

AI 에이전트 협업 설계

1인 개발자가 만드는 멀티에이전트 시스템

AI 에이전트 협업 설계

1인 개발자가 만드는 멀티에이전트 시스템

ThakiCloud (Hyojung Han)

Contents

1	왜 에이전트 하나가 아니라 여러 개인가	1
2	역할 할당: 에이전트에게 일을 나누는 기술	4
3	통신 프로토콜: 에이전트가 서로 대화하는 법	7
4	장애 격리와 회복: 하나가 무너져도 시스템은 살아야 한다	10

1 왜 에이전트 하나가 아니라 여러 개인가

1.1 단일 에이전트의 현실적 한계

AI 에이전트 하나를 운영하는 일은 생각보다 어렵지 않다. 프롬프트를 잘 쓰고 도구를 달아주면 원하는 결과를 낸다. 문제는 요구 사항이 늘어날 때 생긴다. 하나의 에이전트에게 여러 역할을 동시에 맡기면 그 에이전트의 문맥은 점점 무거워진다. 초기 프롬프트에 적어둔 지시는 500번째 메시지에 이르렀을 때쯤이면 거의 남아 있지 않다. 에이전트는 자신이 무엇을 우선해야 하는지 조금씩 잃어버린다.

단일 에이전트가 겪는 문제는 구체적으로 세 갈래로 나뉜다. 먼저 문맥 포화다. 에이전트의 작업 메모리에는 늘 가장 최근 대화가 올라오는데, 여기에 이전 대화의 핵심 결론까지 더해지면 곧 포화 상태에 이른다. 새 정보를 넣으려면 기존 정보를 밀어내야 한다. 다음은 역할 혼용이다. 한 에이전트가 데이터도 분석하고 코드도 쓰고 이메일까지 보내면, 그 사이를 오가는 데 에너지를 쓰게 된다. 역할마다 최적의 프롬프트가 다른데 하나의 시스템 프롬프트에 전부 담으려 하면 어느 하나는 희생될 수밖에 없다. 마지막은 병렬 처리 불가능이다. 에이전트는 본질적으로 하나의 대화 스레드를 순차적으로 처리한다. 서로 독립적인 세 가지 작업을 동시에 처리하고 싶어도 물리적으로 한 에이전트 안에서는 불가능하다.

1.2 멀티에이전트 아키텍처가 풀어주는 것

여러 에이전트를 서로 협력시키면 이 문제들이 각각 다른 방식으로 풀린다. 역할이 분리되어 있으면 각 에이전트의 문맥은 가볍게 유지된다. 데이터 분석 에이전트는 분석 관련 정보만 기억하면 되고, 이메일 발송 에이전트는 이메일 형식과 수신자 정보만 신경 쓰면 된다. 서로의 문맥을 침범하지

않으니 포화도 더디게 온다.

역할 혼용 문제는 각 에이전트가 하나의 역할만 맡도록 설계하면 자연스럽게 풀린다. 데이터 분석 에이전트는 분석만 하고, 코드 생성 에이전트는 코드만 쓰고, 검토 에이전트는 출력을 검증한다. 전문 분야가 명확하니 프롬프트도 그에 맞춰 최적화할 수 있다.

병렬 처리는 구조적으로 지원된다. 세 개의 에이전트가 각자 독립적인 작업을 맡으면 세 가지 일을 동시에 처리할 수 있다. 하나가 느려지거나 멈춰도 나머지 둘은 정상적으로 진행된다. 전체 시스템의 처리량은 하나로 묶여 있을 때보다 높아진다.

1.3 장애 격리라는 이름의 가치

멀티에이전트 시스템의 가장 실질적인 이점은 장애 격리다. 하나의 에이전트가 잘못된 출력을 내놓거나 무한 루프에 빠지거나 외부 API 호출에서 실패하더라도 다른 에이전트에는 영향이 번지지 않는다. 문제가 생긴 에이전트를 격리하고 나머지가 정상 가동 중인 작업을 이어받으면 된다. 단일 에이전트였다면 하나의 장애가 곧 전체 작업 실패로 이어졌을 것이다.

이것은 단순한 기술적 장점에 그치지 않는다. 1인 개발자가 밤사이 무인 파이프라인을 돌릴 때, 에이전트 하나가 멈춰 있어도 나머지는 계속 움직여주길 바라게 된다. 아침에 와서 문제가 있는 로그 하나만 확인하고 그 부분만 수선하면 끝난다. 전부 멈추어 있었다면 처음부터 다시 시작해야 했을 것이다.

1.4 1인 개발자가 멀티에이전트를 고려하는 시점

모든 프로젝트에 멀티에이전트가 필요한 것은 아니다. 다만 고려해야 할 신호는 비교적 뚜렷하다. 우선 같은 유형의 작업을 반복 처리하고

있다는 느낌이 들 때다. 매일 비슷한 형식의 리포트를 세 개 이상 조합해서 만들어야 한다면, 각 단계를 각각의 에이전트에게 나눠 맡기는 편이 효율적이다. 다음으로 작업이 여러 단계로 나뉘고 앞 단계의 출력이 뒷 단계의 입력으로 들어가는 체인 형태를 띠 때다. 이 체인이 길어질수록 단일 에이전트에서 프롬프트 체이닝을 유지하는 비용도 커진다. 마지막으로 작업 중 하나가 실패해도 나머지는 성공해야 하는 상황일 때다. 이 조건이 자주 등장한다면 멀티에이전트 구조를 검토할 시점이다.

구체적 기준은 이렇게 정리할 수 있다. 독립적인 작업이 두 개 이상 동시에 진행되어야 한다면 멀티에이전트를 고려한다. 단계 간 의존성이 명확하고 한 단계의 결과가 다음 단계에 직접 영향을 준다면, 단일 에이전트에 체이닝하는 편이 오히려 단순하다. 하지만 체인이 네 단계 이상으로 길어지거나 두 단계 이상에서 병렬 처리가 가능해 보인다면 구조를 나누는 쪽이 맞다.

1.5 이 책이 다루는 것

이 책은 1인 개발자가 멀티에이전트 시스템을 실제로 설계하고 운영하는데 필요한 내용을 다룬다. 개념 설명에서 그치지 않고 각 장마다 구체적인 아키텍처 패턴과 코드 수준의 구현 방법을 제시한다. 역할 할당, 통신 프로토콜, 갈등 해결, 공동 의사결정, 장애 격리라는 다섯 가지 주제를 중심으로 프로덕션에서 동작하는 시스템을 만드는 기술을 익힌다.

예제 코드는 Python으로 작성하며, OpenAI Assistant API나 Anthropic의 Claude API 같은 특정 클라우드 API에 의존하지 않는 범용 패턴을 설명한다. 기본 개념은 어떤 에이전트 프레임워크를 쓰더라도 적용할 수 있도록 구성했다.

2 역할 할당: 에이전트에게 일을 나누는 기술

2.1 역할 정의의 핵심 원칙

멀티에이전트 시스템을 설계할 때 가장 먼저 해야 하는 일은 역할을 정의하는 것이다. 이 과정에서 놓치기 쉬운 원칙이 세 가지 있다. 하나는 단일 책임이다. 에이전트 하나가 서로 다른 영역을 두 개 이상 떠맡으면 경계가 흐려진다. 다음은 명확한 경계다. 역할 사이에 회색지대가 많으면 에이전트는 자기 일이 무엇인지 판단하는 데 에너지를 소모한다. 마지막은 최소 노출이다. 각 에이전트가 다른 에이전트의 내부 구현까지 알 필요는 없다. 출력 계약만으로도 충분히 협력할 수 있다.

이 원칙을 구체적으로 적용하는 방법을 살펴보자. 데이터 수집 에이전트와 분석 에이전트가 있다고 하자. 데이터 수집 에이전트의 책임은 명확하다. 주어진 검색어나 소스에서 관련 정보를 가져오는 일이다. 분석 에이전트의 책임도 마찬가지로 분명하다. 수집된 정보를 바탕으로 판단을 내리는 일이다. 둘 사이 경계가 명확하기 때문에, 수집 에이전트가 어떤 방식으로 데이터를 가져오든 분석 에이전트는 출력 형식만 알면 된다.

2.2 시스템 프롬프트에 역할을 박아 넣는 방법

역할을 정의했다면 그 역할을 에이전트의 시스템 프롬프트에 고정해야 한다. 여기서 흔히 저지르는 실수는 역할 설명을 너무 넓게 쓰는 것이다. “당신은 데이터 분석 전문가입니다”라는 문장은 역할이 아니라 영역을 가리킬 뿐이다. 구체적으로 무엇을 하는지, 어떤 출력을 기대하는지, 어떤 상황에서 어떻게 행동해야 하는지를 적어야 한다.

좋은 역할 정의의 예시를 하나 보자. “당신의 역할은 웹 검색 결과를 읽고,

그 내용에서 사실 확인이 필요한 핵심 주장을 추출하는 것이다. 출력은 각 주장마다 한 줄의 문장과 출처 URL로 구성된다. 모호하거나 출처 없는 주장은 '확인불가'로 표시한다.” 이 정의에는 역할 범위, 출력 형식, 예외 처리가 모두 담겨 있다. 프롬프트를 이렇게 구체적으로 쓰면 에이전트는 자기가 할 일을 명확하게 이해한다.

입력과 출력의 계약도 명시해야 한다. 각 에이전트가 어떤 형식의 입력을 받는지, 어떤 형식의 출력을 내보내는지를 시스템 프롬프트에 적어두면, 다른 에이전트나 오케스트레이터가 결과를 가공하지 않고 바로 다음 단계에 전달할 수 있다.

2.3 에이전트 간 데이터 전달 설계

역할을 분리하고 나면 다음 문제는 에이전트끼리 데이터를 어떻게 주고받느냐다. 크게 세 가지 방식이 있다. 먼저 공유 컨텍스트 방식이다. 모든 에이전트가 하나의 공유 메모리나 파일에 접근해 필요한 정보를 읽고 쓴다. 구현은 간단하지만 동시 접근 문제가 생기거나 어떤 에이전트가 언제 데이터를 읽었는지 추적하기 어려워진다. 작은 규모의 시스템이나 초기 프로토타입에서만 권할 만하다.

다음은 메시지 큐 방식이다. 각 에이전트가 자신의 출력을 메시지 큐에 전달하면 다음 에이전트가 그 큐에서 데이터를 가져간다. 생산자와 소비자가 분리되어 있어 동시 접근 문제가 해소되고, 어느 에이전트가 어느 데이터를 처리했는지 로그로도 추적할 수 있다. 구현 복잡도는 올라가지만 확장성과 장애 격리 측면에서는 유리하다. 실무에서 가장 많이 쓰는 방식이기도 하다.

마지막은 직접 전달 방식이다. 에이전트 A가 에이전트 B를 직접 호출해 입력을 전달하고 출력을 받는다. 동기적 협력에는 적합하지만 A와 B가 강하게 결합한다는 단점이 있다. B의 인터페이스가 바뀌면 A도 함께 수정해야 하고, 상태 관리도 복잡해진다. 가능하면 피하는 게 나은

패턴이다.

2.4 역할 경계가 흐려질 때의 대처

실무에서 역할 경계가 처음 설계한 대로 유지되지 않는 경우가 많다. 분석 에이전트에게 데이터 정제까지 요청하는 것이 자연스러워지고, 어느새 분석 에이전트가 데이터 정제 역할까지 수행하고 있었다. 이런 상황에서는 두 가지 대처가 가능하다.

하나는 역할을 재정의해 에이전트를 나누는 방법이다. 데이터 정제 에이전트를 새로 만들어 분석 에이전트와 분리하면 된다. 이 방식이 결국 더 올바른 구조로 이어진다. 다른 하나는 경계를 협상하는 방법이다. 모든 역할을 분리하는 것이 과하다고 판단되면 주요 두 가지 역할만 유지하고 나머지는 프롬프트로 관리한다. 중요한 것은 의도적으로 선택하느냐다. 방치한 채 저절로 경계가 흐려지게 두기보다는, 통합이나 분리를 명시적으로 결정해야 시스템을 예측 가능하게 유지할 수 있다.

3 통신 프로토콜: 에이전트가 서로 대화하는 법

3.1 동기 통신과 비동기 통신

에이전트 간 통신을 설계할 때 가장 먼저 마주치는 선택은 동기나 비동기이다. 동기 통신은 에이전트 A가 에이전트 B를 호출하고 B의 응답을 기다린 뒤에야 다음 행동으로 넘어간다. 비동기 통신은 A가 메시지를 보내고 바로 다른 일을 할 수 있다. B가 언제 처리하는지는 A가 알지 못한다.

동기 통신이 적합한 경우부터 보자. 작업 A의 출력이 작업 B의 입력으로 반드시 필요한 체인 형태에서는 동기 통신이 자연스럽다. B의 결과를 받아야 다음 단계로 넘어갈 수 있으니 A의 완료를 기다리는 편이 맞다. 다만 이 패턴이 연속으로 이어지면 체인이 길어질수록 전체 처리 시간은 가장 느린 단계가 좌우한다. 체인의 세 번째 단계가 아무리 빨라도 첫 번째 단계가 느리면 전체가 느려진다.

비동기 통신은 독립적인 단계 사이에 적합하다. 에이전트 A가 데이터를 수집하고 에이전트 B와 C가 각각 그 데이터를 다른 방식으로 분석하는 경우를 생각해보자. A의 수집이 끝나면 B와 C는 동시에 분석을 시작할 수 있다. A가 B의 분석 완료를 기다릴 필요는 없다. 각자 자기 속도로 처리하면 된다.

3.2 공유 컨텍스트 기반 통신

실무에서 가장 많이 쓰이는 통신 패턴은 공유 컨텍스트 기반이다. 중앙 저장소에 각 에이전트가 읽고 쓸 수 있는 데이터를 둔다. 에이전트 A는

수집한 데이터를 중앙 저장소에 쓰고, 에이전트 B는 그 저장소에서 데이터를 읽어 분석을 시작한다. 직접 호출이 없으니 결합도가 낮아지고, 에이전트를 추가하거나 제거해도 기존 에이전트에 미치는 영향이 적다.

구체적인 구현 형태로는 JSON 파일, 데이터베이스, Redis 등이 있다. 파일 기반은 구현이 가장 간단하지만 동시 쓰기 충돌과 최신 데이터 읽기 보장이 필요하다. 데이터베이스는 트랜잭션으로 동시성을 관리해 더 안정적이나 스키마 관리와 마이그레이션 비용이 든다. Redis는 인메모리라 빠른 대신 서버가 재시작되면 데이터가 사라지므로 영속성이 필요한 데이터에는 맞지 않는다. 선택 기준은 단순하다. 데이터 규모가 작고 에이전트 수가 적으면 파일로 시작하고, 동시성 문제와 성능 병목이 실제로 나타나기 시작하면 그때 데이터베이스나 Redis로 옮긴다.

3.3 의사결정 위임 구조

멀티에이전트 시스템에서 가장 설계하기 어려운 부분은 각 에이전트에게 어느 수준까지 판단을 맡길 것인가다. 모든 판단을 중앙에 맡기면 중앙 에이전트가 병목이 되고, 모든 판단을 각 에이전트에 맡기면 시스템 전체의 일관성이 무너진다.

균형 잡힌 구조는 계층적 위임이다. 각 에이전트는 자기 역할 범위 안의 판단을 스스로 내린다. 자기 역할을 벗어나거나 두 가지 판단이 서로 충돌하는 예외적인 상황에서는 상위 레벨에 질의한다. 이 계층 구조가 깊어질수록 각 에이전트가 어느 수준까지 결정을 내릴 수 있는지를 명확히 해둬야 한다.

구체적인 기준을 세워두면 도움이 된다. 에이전트 A는 입력 데이터 중 어느 부분을 쓸지, 분석 방법 중 무엇을 먼저 적용할지, 출력 품질이 충분한지를 스스로 판단할 수 있다. 반면 분석 대상이 애초에 자기 역할에 맞는지, 상충하는 두 분석 결과 중 어느 쪽이 맞는지, 시스템 전체 목표가 바뀌었을 때 새 방향에 맞춰 조정하는 일은 상위 레벨에 맡긴다.

3.4 통신 실패에 대한 설계

에이전트 간 통신은 실패할 수 있다. 네트워크 문제, 대상 에이전트의 응답 지연, 데이터 형식 불일치 등 원인은 다양하다. 실패에 대한 설계가 없으면 통신 장애 하나로 시스템 전체가 멈춰버린다.

기본적인 대처는 세 단계로 나뉜다. 가장 먼저는 재시도다. 통신이 실패하면 일정 횟수만큼 다시 시도하고, 지수적 백오프를 적용해 과부하를 막으면서 회복을 노린다. 다음은 폴백이다. 재시도까지 실패하면 대안 출력으로 넘어가는데, 캐시된 이전 결과를 쓰거나 보수적인 기본값을 내보낸다. 마지막은 격리다. 통신 장애를 일으킨 에이전트를 시스템에서 잠시 떼어내고 나머지 에이전트는 정상적으로 돌아가게 둔다. 떼어낸 에이전트는 백그라운드에서 복구를 시도한다.

이 세 단계를 모두 갖추면 통신 장애 하나가 시스템 전체의 실패로 번지지 않는다. 재시도가 대부분을 복구하고 폴백이 최소한의 기능을 지키며 격리가 영향을 최소화한다.

4 장애 격리와 회복: 하나가 무너져도 시스템은 살아야 한다

4.1 결합도를 낮추는 구조적 설계

장애 격리의 핵심은 에이전트 간 결합도를 낮추는 것이다. 결합도가 높다는 것은 A가 B의 내부 상태나 구현 방식을 알고 있다는 뜻이다. B의 인터페이스가 조금만 바뀌어도 A도 함께 수정해야 한다. 더 심각한 문제는 결합도가 높으면 B의 장애가 A로 직접 전파된다는 것이다.

결합도를 낮추는 구체적인 방법부터 보자. 첫째는 인터페이스 추상화다. 에이전트끼리 직접 메시지를 호출하지 말고 중계 인터페이스를 둔다. A가 B를 호출할 때는 항상 브로커를 거치게 한다. 브로커가 에이전트 간 통신을 관리하므로 에이전트가 교체되어도 브로커 설정만 바꾸면 된다. 에이전트 간 직접 참조가 사라지는 만큼 결합도도 낮아진다.

둘째는 입력 검증이다. 각 에이전트가 받은 입력을 처리하기 전에 항상 검증한다. 예상한 형식이 아닌 데이터가 들어오면 거절하거나 기본값으로 대체한다. 이렇게 하면 한 에이전트가 잘못된 출력을 내보내도 그것을 받는 에이전트가 바로 무너지지 않는다.

셋째는 타임아웃 설정이다. 에이전트 호출에는 항상 타임아웃을 건다. 응답이 일정 시간 내에 없으면 실패로 처리하고 폴백 경로로 이동한다. 타임아웃이 없으면 응답 없는 에이전트 때문에 전체 시스템이 무한 대기 상태에 빠질 수 있다.

4.2 타임아웃과 폴백의 실전 구현

폴백 전략은 에이전트의 역할에 따라 다르게 설계한다. 데이터 수집 에이전트의 폴백은 이전에 수집한 캐시 데이터를 쓰는 것이다. 완전한 데이터가 아니어도 분석에 필요한 최소한의 정보만 있으면 분석 에이전트는 보수적인 결론을 낼 수 있다.

분석 에이전트의 폴백은 기본값을 출력하는 것이다. 분석을 끝낼 수 없으면 “분석 불가, 수동 확인 필요”라는 결과를 내보내고 건너뛴다. 이 결과를 받은 상위 에이전트나 오케스트레이터가 수동 처리를 요청하는 식으로 대응한다.

실제 코드에서 타임아웃과 폴백을 구현하는 구조를 보자. 에이전트 호출을 감싸는 함수를 만들고 그 함수 안에서 재시도 로직, 타임아웃, 폴백을 한 번에 처리한다. 이렇게 하면 에이전트를 호출하는 부분마다 매번 똑같은 장애 처리 로직을 반복하지 않아도 된다.

재시도 횟수는 타임아웃 시간과 함께 에이전트의 중요도에 따라 조정한다. 핵심 역할의 에이전트는 재시도 횟수를 늘리고, 보조적 역할은 줄인다. 다만 재시도 횟수가 아무리 많아도 결국에는 실패로 처리해야 한다. 무한 재시도는 시스템 전체를 차단한다.

4.3 에이전트 재시작과 상태 복구

실패한 에이전트를 다시 시작할 때 고려해야 할 것은 무엇을 복구하느냐이다. 크게 두 가지가 있다. 첫째는 휘발성 상태다. 에이전트가 메모리에만 가지고 있던 진행 중 작업 데이터는 복구 대상이 아니다. 재시작하면 그냥 새 작업부터 하면 된다. 둘째는 영속성 상태다. 에이전트가 중앙 저장소나 파일에 기록해둔 결과물은 그대로 남는다. 다른 에이전트가 그 결과물을 쓰고 있었다면, 재시작 후에도 이어서 작업할 수 있다.

영속성 상태를 복구할 때 중요한 것은 idempotency, 즉 멱등성이다. 같은

입력이면 같은 출력을 내는 동작을 여러 번 반복해도 결과가 달라지지 않아야 한다. 에이전트는 “세 번째 스텝부터 다시 시작”하는 것이 아니라 “첫 스텝부터 다시 시작하되, 이미 완료된 결과는 건너뛰고” 동작해야 한다. 이렇게 하면 재시작 후 같은 데이터를 두 번 처리해서 중복 결과가 나오는 것을 막는다.

4.4 장애 감지와 알림 설계

장애 격리가 작동하려면 장애가 발생했음을 먼저 감지해야 한다. 감지 방식은 세 가지다. 첫째는 응답 없음 감지다. 에이전트가 정해진 시간 내에 응답하지 않으면 장애로 판단한다. 가장 기본적인 방식이고, 대부분의 통신 library가 기본 기능으로 제공한다.

둘째는 명시적 health check다. 주기적으로 각 에이전트에 ping을 보내 응답이 오는지 확인한다. 응답 없음 감지보다 빠르게 알 수 있지만, health check를 위한 별도 통신이 발생한다. 에이전트 수가 늘어나면 그만큼 오버헤드도 커진다.

셋째는 결과물 기반 감지다. 에이전트가 일정한 간격으로 상태를 중앙 저장소에 기록하고 다른 에이전트나 모니터가 그 상태를 확인한다. 에이전트 스스로 “나는 살아있다”고 알리는 방식이라 응답 없음 감지보다 정확하다. 구현은 복잡해지지만 장애 감지 정확도가 중요한 시스템에서는 효과적이다.

감지한 장애는 반드시 알림으로 전달해야 한다. 1인 개발자라면 시스템이 멈추어도 아침에 와서 확인할 때까지 장애를 못 알아챌 수 있다. 슬랙이나 메일, 문자 같은 즉시 알림 채널로 장애를 알려주면 문제가 커지기 전에 대응할 수 있다. 알림의 세밀도는 조정해야 한다. 매번 모든 에이전트의 상태를 알려주면 노이즈만 늘어난다. 핵심 역할 에이전트의 장애와 일정 시간 이상 지속된 장애만 알리도록 필터링한다.